

Guia do desafio

Folhagem & Cia

Tudo o que você precisa saber para construir, do zero, a API de uma loja de plantas: conceitos, ambiente, passo a passo, armadilhas e o que vamos avaliar.

Node.js · Express · TypeScript · Prisma · PostgreSQL · Docker

Leia este guia com calma antes de começar a programar. Ele foi escrito para responder às dúvidas mais comuns, mas os instrutores estão aqui para ajudar: pergunte sempre que travar.

Sumário

1 **Boas-vindas**
o que você vai construir · como usar este guia

2 **Conceitos básicos**
API · CRUD · requisição e resposta · status HTTP · JSON · async/await

3 **Preparando o ambiente**
o que instalar · rodando o projeto · comandos do dia a dia · deu erro?

4 **Entendendo o projeto**
pastas · caminho de uma requisição · lendo o CRUD de usuário

5 **O desafio**
a entidade Planta · requisitos obrigatórios · bônus

6 **Passo a passo para desenvolver**
model · migration · repository · controller · rotas · filtros · venda · valor do estoque · testes

7 **Boas práticas**
nomes · commits · código limpo

8 **Armadilhas comuns**

9 **Entrega e avaliação**
checklist final · como entregar · critérios · pedindo ajuda

10 **Glossário**

1 Boas-vindas

A **Folhagem & Cia** é uma loja de plantas que vende pela internet. Hoje o controle de tudo é feito numa planilha, e isso já deu problema: a loja vendeu uma samambaia que não existia mais no estoque. 😬

A sua missão é construir a **API** que vai substituir essa planilha. Com ela, a loja vai conseguir cadastrar plantas, consultar o catálogo, editar informações, remover plantas, registrar vendas sem vender o que não tem e saber quanto dinheiro está parado no estoque.

O que você vai aprender

- Como uma API recebe pedidos e devolve respostas (requisição e resposta HTTP).
- Como montar um **CRUD** completo: criar, ler, atualizar e deletar.
- Como modelar uma tabela no banco com o **Prisma** e versionar mudanças com **migrations**.
- Como organizar o código em camadas (**rotas** → **controller** → **repository**).
- Como validar dados e responder com o **status HTTP** certo em cada situação.
- Como implementar regras de negócio de verdade, como uma venda que respeita o estoque.

Como usar este guia

Se você...	Leia
nunca fez uma API	Todas as partes, na ordem. A parte 2 explica os conceitos do zero.
já sabe o que é uma API	Comece pela parte 3 (ambiente) e pela parte 4 (o projeto).
está pronto para codar	Partes 5 e 6: o desafio e o passo a passo.
travou em algum erro	Seção 3.4 (erros de ambiente) e parte 8 (armadilhas).
está quase entregando	Parte 9: checklist final e critérios de avaliação.

> 💡 **O segredo do desafio**
> O repositório já vem com um **CRUD de usuários pronto e funcionando**. Ele é o seu modelo. Quase tudo que você precisa fazer nas plantas tem um equivalente nos usuários: abra os dois lado a lado e vá adaptando. Entender o exemplo vale mais do que copiá-lo.

2 Conceitos básicos

2.1 O que é uma API?

Pense num restaurante. Você (o **cliente**) não entra na cozinha: você faz o pedido ao **garçom**, que leva até a **cozinha** e volta com o prato. Numa aplicação:



A **API** é o garçom: recebe o pedido (a **requisição**), confere se faz sentido, busca ou grava no banco e devolve uma **resposta**. O cliente nunca fala direto com o banco.

2.2 O que é CRUD?

CRUD são as quatro operações básicas que quase todo sistema faz com os seus dados:

Letra	Operação	Método HTTP	Função do Prisma	Exemplo
C	Create (criar)	`POST`	`create`	Cadastrar uma planta
R	Read (ler)	`GET`	`findMany`, `findUnique`	Listar o catálogo
U	Update (atualizar)	`PATCH`	`update`	Mudar o preço
D	Delete (deletar)	`DELETE`	`delete`	Remover do catálogo

> 📘 **PATCH ou PUT?**
>
> Os dois atualizam. O `PUT` substitui o recurso inteiro (você manda todos os campos). O `PATCH` altera só o que você mandar. Neste projeto usamos **PATCH**: se o corpo tiver só `{ "preco": 9990 }`, só o preço muda.

2.3 Anatomia de uma requisição

Toda requisição tem um **método** e uma **URL**, e pode levar dados em três lugares diferentes:

```
PATCH  http://localhost:3001/plantas/3?avisar=true
Content-Type: application/json

{ "preco": 9990 }
```

Onde	No exemplo	Como ler no Express	Para que serve
Params (parâmetros da rota)	`3`	`req.params.id`	Identificar *qual* recurso: a planta 3
Query string (depois do `?`)	`avisar=true`	`req.query.avisar`	Opções e filtros: `?petFriendly=true`

Body (corpo)	`{ "preco": 9990 }`	`req.body.preco`	Os dados enviados: o que criar ou mudar

Para o Express saber que `/plantas/3` tem um parâmetro, a rota é declarada com dois-pontos: `/plantas/:id`. O nome depois dos dois-pontos vira a chave em `req.params`.

> ⚠️ **Params e query string são sempre texto**
>
> Em `/plantas/3`, o `req.params.id` é a string `"3"`, não o número `3`. Em `?petFriendly=true`, o valor é a string `"true"`, não o booleano `true`. Você vai precisar converter (seções 6.4 e 6.6). Já o **body**, por ser JSON, mantém os tipos.

2.4 Anatomia de uma resposta e os status HTTP

A resposta tem um **status** (um número que resume o que aconteceu) e, geralmente, um **corpo** em JSON. O status é a primeira coisa que o cliente olha, por isso escolher o certo importa muito.

Status	Quando usar	Exemplo no desafio
`200 OK`	Deu certo e há algo para devolver	Listar, buscar, editar, vender
`201 Created`	Algo novo foi criado	Cadastrar planta
`204 No Content`	Deu certo e não há corpo para devolver	Deletar planta
`400 Bad Request`	O cliente mandou dados faltando ou inválidos	Preço negativo, venda sem estoque
`404 Not Found`	O recurso pedido não existe	Buscar a planta 999
`409 Conflict`	Viola uma regra de unicidade	Cadastrar um dado único que já existe
`500 Internal Server Error`	Algo inesperado quebrou do nosso lado	O banco caiu

> 💡 **Regra de bolso**
>
> **2xx**: deu certo. **4xx**: a culpa é de quem pediu (o cliente precisa corrigir algo). **5xx**: a culpa é nossa (o servidor falhou). Por isso o `catch` genérico deve responder **500**, nunca 400.

2.5 JSON: o formato dos dados

JSON é o formato de texto que a API usa para receber e enviar dados. Preste atenção aos tipos:

```
{
  "nome": "Jiboia",          // string: sempre entre aspas duplas
  "preco": 3490,             // número: SEM aspas
  "petFriendly": false,      // booleano: true ou false, SEM aspas
  "luminosidade": "SOMBRA"   // enum: chega como string
}
```

> ⚠️ **`"preco": "3490"` não é a mesma coisa que `"preco": 3490`**
>
> O primeiro é texto; o segundo é número. Uma API bem validada recusa o primeiro com 400. Quando for testar, confira se está mandando os tipos certos. E lembre: JSON não aceita comentários. Os `//` acima são só explicação.

2.6 `async` / `await` e `try` / `catch`

Falar com o banco leva tempo, então as funções do Prisma devolvem uma *Promise* (uma promessa de resultado). O `await` espera o resultado chegar, e só pode ser usado dentro de uma função `async`.

```
export const readAllUsers = async (req: Request, res: Response) => {
  try {
    const users = await UserRepository.findAllUsers(); // espera o banco responder
    return res.status(200).json(users);
  } catch (error) {
    console.error(error);                                  // mostra o erro no terminal
    return res.status(500).json({ message: "Erro ao buscar usuários." });
  }
};
```

O `try` / `catch` garante que, se algo der errado lá dentro (o banco cair, por exemplo), a API responde com 500 e uma mensagem, em vez de deixar o cliente esperando para sempre.

> 🚫 **Esqueceu o `await`?**
>
> Sem ele, a variável recebe a *Promise*, não os dados. O resultado costuma ser uma resposta `{}` vazia ou um `if (!planta)` que nunca é verdadeiro. Se algo "não faz sentido", confira os `await`.

3 Preparando o ambiente

3.1 O que instalar

Ferramenta	Para quê	Como conferir
Git	Baixar o projeto e versionar o seu código	`git --version`
Node.js 20 ou mais novo	Rodar JavaScript/TypeScript fora do navegador	`node -v`
Yarn	Instalar as dependências (`npm install -g yarn`)	`yarn -v`
Docker Desktop	Rodar a API e o banco sem instalar o PostgreSQL	`docker --version`
VS Code	Editor de código	
Insomnia, Postman ou Thunder Client	Fazer requisições para testar a API	

> 💡 **Extensões do VS Code que ajudam muito**
>
> **Prisma** (cores e autocompletar no `schema.prisma`) e **Thunder Client** (testar a API sem sair do editor).

3.2 Rodando o projeto pela primeira vez

Abra o terminal na pasta onde quer guardar o projeto e siga a ordem:

```
# 1. Crie o SEU repositório (botão "Use this template" no GitHub) e baixe ele
git clone <link-do-seu-repositório>
cd <nome-do-seu-repositório>

# 2. Confira que o projeto aponta para o SEU repositório (e não para o do CITi)
git remote -v

# 3. Crie o arquivo de configuração a partir do modelo
cp .env.example .env            # no PowerShell: copy .env.example .env

# 4. Instale as dependências
yarn

# 5. Abra o Docker Desktop e espere ele ficar pronto. Depois suba a API e o banco:
docker compose up -d --build

# 6. Crie as tabelas no banco
yarn migration
```

No passo 6, o Prisma cria a tabela do User com a migration que já vem no projeto, sem pedir nome.

Para conferir se deu tudo certo, abra `http://localhost:3001` no navegador. Você deve ver:

```
{ "message": "API do treinamento de CRUD do CITi 🚀" }
```

> **📘 O que o Docker está fazendo?**
>
> Ele sobe dois "computadorzinhos" isolados, chamados **containers**: um com a sua API e outro com o banco PostgreSQL. A pasta do projeto é compartilhada com o container da API, então **quando você salva um arquivo, a API reinicia sozinha**. Você não precisa rodar nada de novo a cada mudança.

3.3 Comandos do dia a dia

Comando	O que faz
`docker compose up -d`	Sobe a API e o banco (use no começo do dia)
`docker compose logs -f server`	Mostra os logs da API em tempo real. **É aqui que aparecem os seus erros e `console.log`** (`Ctrl+C` para sair)
`docker compose down`	Desliga tudo (os dados do banco continuam salvos)
`yarn migration`	Cria uma migration a partir do `schema.prisma` e aplica no banco
`yarn studio`	Abre o Prisma Studio no navegador para ver e editar os dados do banco
`yarn typecheck`	Procura erros de tipo no TypeScript em todo o projeto

3.4 Deu erro? Comece por aqui

Mensagem ou sintoma	O que fazer
`failed to connect to the docker API` ou `Cannot connect to the Docker daemon`	O Docker Desktop está fechado. Abra e espere o ícone ficar verde.
`port is already allocated`	Outra coisa já usa a porta 3001 ou a 5432 (por exemplo, um PostgreSQL instalado na sua máquina). Troque `SERVER_PORT` ou `DATABASE_PORT` no `.env` (ex.: 3002 ou 5433) e rode `docker compose up -d` de novo.
`Can't reach database server` ao rodar `yarn migration`	O banco não está de pé. Rode `docker compose up -d` e confira com `docker compose ps`.
`Property 'planta' does not exist on type 'PrismaClient'`	O Prisma ainda não conhece o seu model novo. Rode `yarn migration` (ou `yarn generate`) e, no VS Code, `Ctrl+Shift+P` → *TypeScript: Restart TS Server*.
`'cp' não é reconhecido` no Windows	No PowerShell, use `copy .env.example .env`.
A requisição fica carregando para sempre	Falta um `return res.status(...)` em algum caminho do controller, ou a API caiu. Veja os logs.

`req.body` chega `undefined`	No Insomnia/Postman, o corpo precisa estar como **JSON** (e não Text ou Form).
Qualquer outra coisa estranha	`docker compose logs -f server`: o erro quase sempre está lá, com o arquivo e a linha.

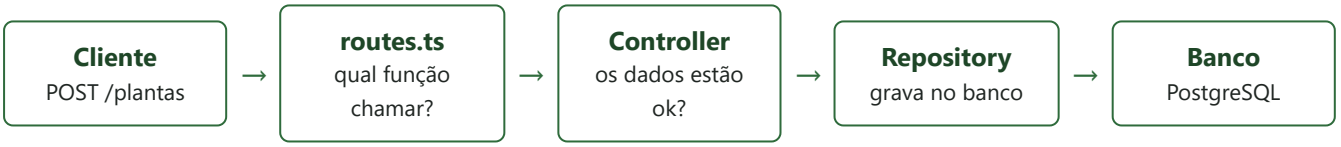
4 Entendendo o projeto

4.1 As pastas

```
treinamento-crud-citi/
├── prisma/
│   ├── schema.prisma        # os models (tabelas) do banco
│   └── migrations/          # o histórico de mudanças do banco (criado pelo yarn migration)
├── src/
│   ├── server.ts            # cria o app Express e liga o servidor
│   ├── routes.ts            # liga cada rota HTTP a uma função do controller
│   ├── database/index.ts    # a conexão com o banco (Prisma Client)
│   ├── controllers/         # recebe a requisição, valida e responde
│   └── repositories/        # a única camada que conversa com o banco
├── .env                     # configurações (portas, senha do banco); não vai para o git
├── docker-compose.yml       # descreve os containers da API e do banco
├── README.md                # como rodar o projeto
└── DESAFIO.md               # o enunciado do desafio
```

No seu desafio, você vai mexer em `prisma/schema.prisma`, em `src/routes.ts` e criar dois arquivos novos: `src/controllers/PlantaController.ts` e `src/repositories/PlantaRepository.ts`.

4.2 O caminho de uma requisição



A resposta faz o caminho de volta: o repository devolve os dados para o controller, que escolhe o status e manda o JSON para o cliente.

Camada	Faz	Não faz
routes	Diz que `POST /plantas` chama `createPlanta`	Nenhuma lógica, nenhuma validação
controller	Lê `req`, valida, decide o status, monta a resposta	Não usa o `prisma` diretamente
repository	Faz as consultas no banco com o Prisma	Não conhece `req` nem `res`, não decide status

> 📘 **Por que separar em camadas?**
>
> Cada arquivo tem uma responsabilidade só, então fica fácil achar as coisas e mudar uma parte sem quebrar outra. Se um dia o banco mudar, só o repository muda. Se a regra de validação mudar, só o controller muda.

4.3 Lendo o CRUD de usuário

Vamos seguir o cadastro de um usuário por todas as camadas. Abra os arquivos no VS Code e acompanhe.

O model, em `prisma/schema.prisma`

```
model User {
  id        Int      @id @default(autoincrement())  // chave primária, gerada automaticamente
  nome      String
  conta     Conta                                   // enum: CPF ou CNPJ
  documento String   @unique                        // não pode repetir
  createdAt DateTime @default(now())                // preenchido na criação
  updatedAt DateTime @updatedAt                     // atualizado a cada edição
}
```

Cada `model` vira uma tabela no banco, e cada linha dentro dele vira uma coluna. O que vem depois do `@` são regras extras.

A rota, em `src/routes.ts`

```
routes.post("/users", UserController.createUser);
routes.get("/users", UserController.readAllUsers);
routes.get("/users/:id", UserController.readUserById);
routes.patch("/users/:id", UserController.updateUser);
routes.delete("/users/:id", UserController.deleteUser);
```

Cinco rotas, cinco funções. Repare no padrão: a URL é o **nome do recurso no plural** (`/users`) e o que muda é o método.

O repository, em `src/repositories/UserRepository.ts`

```
// O repository é a única camada que conversa com o banco.

export const createUser = async (data: Prisma.UserCreateInput) => {
  return await prisma.user.create({
    data: data,
  });
};

export const findUserById = async (id: number) => {
  return await prisma.user.findUnique({
    where: { id: id },
  });
};
```

Funções pequenas, cada uma com uma consulta. O tipo `Prisma.UserCreateInput` é gerado automaticamente pelo Prisma a partir do model: com ele, o TypeScript avisa se você esquecer um campo ou errar um tipo.

O controller, em `src/controllers/UserController.ts`

```
export const createUser = async (req: Request, res: Response) => {
  try {
    const { nome, conta, documento } = req.body;         // ① lê os dados

    // 400 Bad Request: o cliente mandou dados incompletos
    if (!nome || !conta || !documento) {                 // ② valida
      return res.status(400).json({
        message: "Preencha nome, conta e documento.",
      });
    }

    const user = await UserRepository.createUser({       // ③ grava pelo repository
      nome: nome,
      conta: conta,
      documento: documento,
    });

    // 201 Created: algo novo foi criado
    return res.status(201).json(user);                   // ④ responde com o status certo
  } catch (error) {
    console.error(error);
    // 500 Internal Server Error: o problema foi nosso, não do cliente
    return res.status(500).json({ message: "Erro ao criar usuário." });
  }
};
```

Quase todo handler segue esses passos: **ler** → **validar** → **chamar o repository** → **responder**. Nas plantas, antes de chamar o repository entram também as **regras de negócio** (como o estoque na venda). Cada problema tem o seu `return` com um status específico, e o caminho feliz fica no final.

Buscar, editar e deletar: o padrão do "não encontrado"

```
const user = await UserRepository.findUserById(id);

if (!user) {
  return res.status(404).json({ message: "Usuário não encontrado." });
}
```

Antes de editar ou deletar, o controller **busca primeiro**. Se não achar, responde 404. Sem essa checagem, o Prisma lança um erro, que cai no `catch` e vira um 500, o que está errado: a culpa foi de quem pediu um id que não existe.

> 💡 **Experimente antes de codar**
>
> Antes de começar o desafio, use o Insomnia para chamar todas as rotas de `/users`, incluindo os erros: mande um id que não existe, um corpo vazio ou sem a conta. Ver as respostas na prática ajuda muito a entender o código.

5 O desafio

5.1 A entidade `Planta`

Campo	Tipo	Exemplo	Observação
`id`	Int (autoincrement)	`1`	Gerado pelo banco
`nome`	String	`"Costela-de-adão"`	Nome popular
`especie`	String	`"Monstera deliciosa"`	Nome científico
`preco`	Int	`8990`	Em **centavos**: `8990` = R$ 89,90
`estoque`	Int	`12`	Quantidade de vasos disponíveis
`luminosidade`	Enum	`"MEIA_SOMBRA"`	`SOMBRA`, `MEIA_SOMBRA` ou `SOL_PLENO`
`petFriendly`	Boolean	`true`	Se é segura para cães e gatos
`regaACadaDias`	Int	`3`	De quantos em quantos dias regar
`createdAt` / `updatedAt`	DateTime		Preenchidos automaticamente

> 📘 **Por que o preço é em centavos?**
> Números decimais no computador têm erros de arredondamento: em JavaScript, `0.1 + 0.2` dá `0.30000000000000004`. Com dinheiro, isso vira prejuízo. Guardando em centavos (números inteiros), as contas são sempre exatas. Quem mostra "R$ 89,90" na tela é o front-end.

> 📘 **O que é um enum?**
> Um campo que só aceita uma lista fechada de valores. A luminosidade não é um texto livre: é *uma das três* opções. O próprio banco recusa qualquer outro valor, e a sua API deve responder 400 antes disso.

5.2 Requisitos obrigatórios

CRUD

Método	Rota	O que faz	Sucesso
`POST`	`/plantas`	Cadastra uma planta	`201`
`GET`	`/plantas`	Lista todas as plantas	`200`
`GET`	`/plantas/:id`	Busca uma planta pelo id	`200`
`PATCH`	`/plantas/:id`	Edita uma planta	`200`

DELETE	`/plantas/:id`	Remove uma planta	`204`

Regras de negócio

1. Todos os campos são obrigatórios no cadastro.
2. `preco` precisa ser maior que zero.
3. `estoque` não pode ser negativo, mas **pode ser zero** (planta esgotada).
4. `regaACadaDias` precisa ser pelo menos `1`.
5. `luminosidade` só aceita os três valores do enum. Qualquer outro valor deve retornar `400`.
6. Buscar, editar ou deletar um `id` que não existe deve retornar `404`.

Funcionalidades da loja

#	Rota	O que faz
7	GET `/plantas?luminosidade=…&petFriendly=…`	**Filtros opcionais** na listagem. Funcionam sozinhos ou juntos.
8	POST `/plantas/:id/venda`	**Vende** a quantidade do corpo (`{ "quantidade": 2 }`) e diminui o estoque. Sem estoque suficiente: `400` e nada muda.
9	GET `/plantas/estoque/valor-total`	**Quanto vale o estoque**: a soma de `preco × estoque` de todas as plantas, e o total de vasos.

Formato esperado da resposta da funcionalidade 9:

```
{ "valorTotalEmCentavos": 154300, "totalDeVasos": 37 }
```

5.3 Bônus (opcionais)

- 🔎 Busca por nome sem diferenciar maiúsculas e minúsculas: `GET /plantas?nome=costela`.
- ↕️ Ordenação: `GET /plantas?ordem=preco` ou `?ordem=nome`.
- 🏷️ Rota `GET /plantas/esgotadas`, que lista as plantas com estoque zero.
- 🌱 Um script de *seed* que popula o banco com algumas plantas de exemplo.

> ⚠️ **Faça o obrigatório primeiro**
>
> Um CRUD completo, bem validado e com os status certos vale muito mais do que um projeto cheio de bônus pela metade. Só parta para os bônus quando tudo da seção 5.2 estiver funcionando e testado.

6 Passo a passo para desenvolver

Siga a ordem abaixo. A cada passo, **teste e faça um commit** antes de seguir para o próximo.

PASSO 1 O model e o enum

Abra `prisma/schema.prisma` e, abaixo do `User`, crie o enum e o model. A sintaxe de um enum é esta (exemplo com outro tema):

```
enum Tamanho {
  PEQUENO
  MEDIO
  GRANDE
}

model Camiseta {
  id        Int      @id @default(autoincrement())  // id numérico: 1, 2, 3...
  cor       String
  tamanho   Tamanho                                 // usa o enum como tipo
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

Agora é com você: crie o `enum Luminosidade` e o `model Planta` com todos os campos da seção 5.1.

- ☐ Os nomes dos campos estão **exatamente** como no enunciado (`regaACadaDias`, com as maiúsculas no lugar certo)?
- ☐ `preco` é `Int`, e não `Float`?
- ☐ O id é `Int @id @default(autoincrement())`?
- ☐ Tem `createdAt` e `updatedAt`, como no `User`?

PASSO 2 A migration

```
yarn migration
# Enter a name for the new migration: › create_planta_model
```

A migration é um arquivo SQL, gerado em `prisma/migrations/`, que descreve a mudança no banco. Ela também atualiza os tipos do Prisma Client, e é por isso que, depois dela, `prisma.planta` passa a existir no autocompletar.

Confira o resultado com `yarn studio`: a tabela `Planta` deve aparecer.

> 📘 **Migrations são o "git do banco"**
>
> Cada migration é um passo no histórico do banco, e elas **vão para o repositório** junto com o código. Assim, qualquer pessoa que clonar o projeto e rodar `yarn migration` chega no mesmo banco que você.

> 🚫 **Errou o model? Não edite a migration**
>
> Corrija o `schema.prisma` e rode `yarn migration` de novo, com um nome como `fix_planta_preco`. Uma migration que já foi aplicada nunca deve ser editada à mão.

PASSO 3 O repository

Crie `src/repositories/PlantaRepository.ts`. Use o `UserRepository.ts` como modelo. Estas são as funções de que você vai precisar:

Função	O que faz	Prisma
`createPlanta(data)`	Cria uma planta	`create`
`findAllPlantas(filtros)`	Lista, aplicando os filtros que vierem	`findMany` + `where`
`findPlantaById(id)`	Busca uma planta ou devolve `null`	`findUnique`
`updatePlanta(id, data)`	Edita uma planta	`update`
`deletePlanta(id)`	Remove uma planta	`delete`
`venderPlanta(id, quantidade)`	Diminui o estoque (veja o passo 7)	`update`
`calcularValorDoEstoque()`	Soma o valor do estoque (veja o passo 8)	`findMany`

> ⚠️ **O id da planta é número**
>
> No `User` e na `Planta`, o id é um `Int`. Então as funções recebem `id: number`, e o controller precisa converter o `req.params.id` antes de chamar o repository.

> 💡 **Os tipos do Prisma trabalham por você**
>
> Use `Prisma.PlantaCreateInput` e `Prisma.PlantaUpdateInput` como tipo do parâmetro `data`, como o `UserRepository` faz. Eles aparecem depois da migration do passo 2.

PASSO 4 O controller: validações

Crie `src/controllers/PlantaController.ts`. Este é o passo com mais detalhes, então vamos por partes.

Convertendo e validando o id

```
const id = Number(req.params.id);     // "3" vira 3; "abc" vira NaN

if (!Number.isInteger(id) || id <= 0) {
  return res.status(400).json({ message: "O id precisa ser um número inteiro positivo." });
}
```

Se você usa isso em várias funções, vale criar uma função auxiliar no topo do arquivo, em vez de repetir o código.

⚠️ A armadilha mais importante do desafio: `!campo`

No `UserController`, a validação usa `!nome || !conta`, e lá funciona, porque todos os campos são textos que não podem ser vazios. Nas plantas, isso **quebra**. Veja o que o `!` faz com cada valor:

Valor recebido	`!valor`	É um valor válido para a planta?
`undefined` (campo não enviado)	`true`	❌ Não: precisa recusar
`""` (texto vazio)	`true`	❌ Não
`0` (estoque zero)	`true`	✅ **Sim!** Planta esgotada é válida
`false` (petFriendly)	`true`	✅ **Sim!** Planta tóxica para pets é válida

Ou seja: `if (!estoque)` recusaria uma planta com estoque 0, e `if (!petFriendly)` recusaria toda planta que não é pet-friendly. Para saber se um campo **foi enviado**, compare com `undefined`:

```
if (estoque === undefined || petFriendly === undefined) {
  return res.status(400).json({ message: "Preencha todos os campos." });
}
```

Validando os tipos e as regras

Depois de garantir que o campo veio, confira se ele tem o tipo e o valor certos. Estas são as ferramentas do JavaScript para isso:

Verificação	Quando usar
`typeof nome === "string"`	Textos
`nome.trim() !== ""`	Texto que não pode ser vazio ou só espaços
`Number.isInteger(preco)`	Números inteiros (recusa `"8990"`, `89.9` e `NaN`)
`typeof petFriendly === "boolean"`	Booleanos
`Object.values(Luminosidade).includes(luminosidade)`	Enums. Importe com `import { Luminosidade } from "@prisma/client"`

Combinando com as regras: `!Number.isInteger(preco) || preco <= 0` significa "o preço é inválido". Escreva uma checagem dessas para cada regra da seção 5.2, cada uma com uma mensagem clara.

Validação no cadastro x validação na edição

No **POST**, todos os campos são obrigatórios. No **PATCH**, nenhum é: a pessoa manda só o que quer mudar. Mas os campos que *vierem* no PATCH precisam passar pelas mesmas regras: editar o preço para `-10` deve dar 400.

> 💡 **Não escreva as regras duas vezes**
>
> Uma boa saída é criar uma função que valida *só os campos que vieram* (pulando os `undefined`) e usá-la tanto no POST quanto no PATCH. No POST, antes dela, você confere se nenhum campo está faltando.

Não passe o `req.body` direto para o banco

```
// ❌ Evite: o cliente pode mandar campos que você não esperava (id, createdAt...)
await PlantaRepository.createPlanta(req.body);

// ✅ Prefira: você escolhe exatamente o que vai para o banco
const { nome, especie, preco, estoque, luminosidade, petFriendly, regaACadaDias } = req.body;
await PlantaRepository.createPlanta({ nome, especie, preco, estoque, luminosidade, petFriendly,
regaACadaDias });
```

Mensagens com variáveis: use crase

```
"Temos só ${planta.estoque} vasos"   // ❌ aspas duplas: aparece "${planta.estoque}" literalmente
`Temos só ${planta.estoque} vasos`   // ✅ crase (template string): aparece "Temos só 3 vasos"
```

PASSO 5 As rotas

Registre as rotas em `src/routes.ts`, seguindo o padrão das de usuário. Uma regra importante: o Express testa as rotas **na ordem em que foram registradas** e usa a primeira que combinar.

```
// ❌ Ordem errada: "esgotadas" é capturado pelo :id
routes.get("/plantas/:id", PlantaController.readPlantaById);
routes.get("/plantas/esgotadas", PlantaController.readPlantasEsgotadas);  // nunca é alcançada

// ✅ Ordem certa: rotas fixas antes das rotas com parâmetro
routes.get("/plantas/esgotadas", PlantaController.readPlantasEsgotadas);
routes.get("/plantas/:id", PlantaController.readPlantaById);
```

No primeiro caso, `GET /plantas/esgotadas` chama `readPlantaById` com `id = "esgotadas"`. Essa regra só importa para o bônus, mas é um erro clássico que vale conhecer.

PASSO 6 Filtros na listagem

Os filtros chegam pela query string: `GET /plantas?luminosidade=SOMBRA&petFriendly=true` vira `req.query.luminosidade` e `req.query.petFriendly`. Se o filtro não foi enviado, o valor é `undefined`.

Uma propriedade muito útil do Prisma: **condições com valor `undefined` no `where` são ignoradas**. Com isso, uma única consulta resolve todas as combinações:

```
// exemplo com usuários: se nome for undefined, lista todo mundo
prisma.user.findMany({
  where: { nome: nome },
});
```

Os detalhes que você precisa tratar:

- **Converter o booleano.** `req.query.petFriendly` é a string `"true"` ou `"false"`. Passar a string direto para o Prisma dá erro. Converta comparando: `petFriendly === "true"`. Cuidado para não transformar `undefined` em `false`: filtro não enviado deve continuar `undefined`.
- **Validar os valores.** `?luminosidade=LUA` ou `?petFriendly=talvez` devem retornar 400 com uma mensagem clara.

- **Filtro vazio não é erro.** Se nenhuma planta combina com o filtro, a resposta é `200` com uma lista vazia `[]`, e não 404.

> 💡 **Para os bônus de busca e ordenação**
>
> Pesquise na documentação do Prisma por `contains` com `mode: "insensitive"` (busca sem diferenciar maiúsculas) e por `orderBy` (ordenação).

PASSO 7 A venda

`POST` `/plantas/:id/venda` com o corpo `{ "quantidade": 2 }`. Organize o handler nesta ordem:

1. Valide o **id** → se inválido, `400`.
2. Valide a **quantidade**: precisa ser um inteiro maior que zero → se não for, `400`.
3. Busque a planta → se não existir, `404`.
4. Confira se `planta.estoque >= quantidade` → se não, `400` com uma mensagem como "*Estoque insuficiente: temos só 3 vaso(s) de Jiboia.*" e **sem alterar nada**.
5. Diminua o estoque no banco.
6. Responda `200` com a planta atualizada (e, se quiser, uma mensagem de sucesso).

> 💡 **O Prisma sabe decrementar**
>
> Em vez de calcular o novo estoque no código, você pode pedir ao banco: `data: { estoque: { decrement: quantidade } }`. Existe também o `increment`.

> 📘 **Para pensar (não é obrigatório)**
>
> E se duas pessoas comprarem o último vaso *ao mesmo tempo*? As duas podem passar pelo passo 4 antes de qualquer uma chegar ao passo 5, e o estoque fica negativo. Como resolver isso? Pesquise sobre `updateMany` com condição no `where`. Se você resolver, conte no README da sua entrega!

PASSO 8 O valor total do estoque

`GET` `/plantas/estoque/valor-total`. A conta é: para cada planta, `preco × estoque`; depois, some tudo. Exemplo com duas plantas:

Planta	preco	estoque	preco × estoque
Jiboia	3490	10	34900
Calathea	7990	2	15980
Total		**12 vasos**	**50880** (R$ 508,80)

```
{ "valorTotalEmCentavos": 50880, "totalDeVasos": 12 }
```

> ⚠️ **O `aggregate` não resolve sozinho**
>
> O `aggregate` do Prisma consegue somar *uma coluna* (`_sum: { estoque: true }` dá o total de vasos), mas não consegue somar uma *multiplicação* de colunas. O caminho é buscar as plantas (só os campos `preco` e `estoque`, com `select`) e fazer a soma no código, com um `for` ou um `reduce`.

Lembre: se a loja não tiver nenhuma planta, a resposta é `200` com os dois valores zerados, e não um erro.

PASSO 9 Testando tudo

No Insomnia, Postman ou Thunder Client, crie uma coleção chamada "Folhagem & Cia" com uma requisição para cada rota. Nas requisições com corpo, escolha o tipo **JSON**. Corpo de exemplo para o cadastro:

```
{
  "nome": "Costela-de-adão",
  "especie": "Monstera deliciosa",
  "preco": 8990,
  "estoque": 12,
  "luminosidade": "MEIA_SOMBRA",
  "petFriendly": false,
  "regaACadaDias": 7
}
```

Testar só o caminho feliz não basta: **a maior parte da avaliação está nos casos de erro**. Use esta lista:

Teste	Esperado
Cadastrar uma planta válida	`201`
Cadastrar com `estoque: 0` e `petFriendly: false`	`201`
Cadastrar sem um dos campos	`400`
Cadastrar com `preco: 0`, `estoque: -1` ou `regaACadaDias: 0`	`400`
Cadastrar com `luminosidade: "LUA_CHEIA"`	`400`
Cadastrar com `preco: "8990"` (string)	`400`
Listar todas as plantas	`200` + array
Buscar um id que existe / que não existe / `abc`	`200` / `404` / `400`
Editar só o preço (os outros campos não mudam)	`200`
Editar com um valor inválido / um id que não existe	`400` / `404`
Filtrar por luminosidade, por petFriendly e pelos dois juntos	`200`
Filtrar com `?luminosidade=LUA`	`400`
Vender uma quantidade disponível (confira o estoque depois)	`200`
Vender mais do que tem (confira que o estoque **não mudou**)	`400`
Vender `quantidade: 0` / de um id que não existe	`400` / `404`

Consultar o valor total (confira a conta na mão)	`200`
Deletar / buscar a planta deletada / deletar de novo	`204` / `404` / `404`

7 Boas práticas

7.1 Nomes

O quê	Padrão	Exemplo
Variáveis e funções	camelCase	`plantaAtualizada`, `findPlantaById`
Arquivos de controller e repository	PascalCase + sufixo	`PlantaController.ts`
Rotas	minúsculas, plural, hífen	`/plantas`, `/valor-total`
Valores de enum	MAIÚSCULAS com _	`MEIA_SOMBRA`

Prefira nomes que dizem o que a coisa é: `plantaAtualizada` é melhor que `p2`; `camposFaltando` é melhor que `arr`.

7.2 Commits

Faça commits pequenos, um por passo concluído, com mensagens que dizem *o que* mudou:

```
git add .
git commit -m "Cria model Planta e enum Luminosidade"
git commit -m "Adiciona cadastro e listagem de plantas"
git commit -m "Adiciona validações de preço, estoque e luminosidade"
git commit -m "Implementa a rota de venda"
```

Evite mensagens como `"ajustes"`, `"agora vai"` ou `"final final 2"`. O histórico de commits também é avaliado: ele mostra como você pensou.

7.3 Código limpo

- Apague os `console.log` de teste antes de entregar (o `console.error` dentro do `catch` pode ficar).
- Apague imports e variáveis que não são usados. O VS Code mostra esses em cinza.
- Não deixe código comentado "para garantir": o git já guarda o histórico.
- Escreva mensagens de erro claras, em português, que digam *o que* está errado: "O preço precisa ser maior que zero" é melhor que "Dados inválidos".
- Rode `yarn typecheck` antes de entregar. Ele precisa terminar sem erros.

8 Armadilhas comuns

Estes são os erros que mais aparecem em desafios como este. Releia a lista antes de entregar.

Erro	Como aparece	Como evitar
Validar com `!campo`	Estoque 0 ou petFriendly `false` são recusados	Use `campo === undefined` para saber se o campo veio (passo 4)
404 virando 500	Editar ou deletar um id inexistente responde 500	Busque antes de editar ou deletar, e responda 404 se não achar
`catch` respondendo 400	Erro do servidor tratado como erro do cliente	No `catch` genérico, use 500
Venda deixa o estoque negativo	Vender 10 vasos quando tem 3 funciona	Cheque `estoque >= quantidade` antes de vender
Filtro com string	`?petFriendly=true` dá erro no Prisma	Converta `"true"` para `true` (passo 6)
Id sem conversão	O Prisma reclama que recebeu string onde esperava Int	`Number(req.params.id)` e valide
Lista vazia como 404	Filtro sem resultado responde 404	Lista vazia é `200` com `[]`
`if (!lista)` depois do `findMany`	Código que nunca executa	O `findMany` sempre devolve um array, mesmo vazio
`"${variavel}"` com aspas	A mensagem mostra o texto `${variavel}`	Use crase: `` `${variavel}` ``
Esquecer o `return`	Erro "Cannot set headers after they are sent"	Todo `res.status(...)` dentro de um `if` precisa de `return`
Esquecer o `await`	Resposta `{}` ou checagens que nunca funcionam	Toda chamada ao repository leva `await`
Preço como `Float`	Contas com centavos estranhos	Preço em centavos, como `Int`
`req.body` direto no banco	O cliente consegue mandar `id`, `createdAt`...	Desestruture e passe campo a campo
Nomes diferentes do enunciado	`price`, `stock`, `/plants`	Use exatamente os nomes de campos e rotas do enunciado

> 🚫 **"Cannot set headers after they are sent to the client"**
>
> Esse erro quase sempre significa que o código respondeu duas vezes: um `if` mandou o 400, mas, sem `return`, a função continuou e tentou mandar o 201 também. Procure um `res.status(...)` sem `return` na frente.

9 Entrega e avaliação

9.1 Checklist final

- ☐ O projeto sobe do zero com `docker compose up -d --build` e `yarn migration`
- ☐ A migration da `Planta` está commitada na pasta `prisma/migrations`
- ☐ As 5 rotas do CRUD funcionam com os status certos
- ☐ As 6 regras de negócio estão validadas, com mensagens claras
- ☐ Os filtros funcionam sozinhos e juntos
- ☐ A venda nunca deixa o estoque negativo
- ☐ O valor total bate com a conta feita na mão
- ☐ Todos os testes da tabela do passo 9 foram feitos
- ☐ Os nomes de campos e rotas estão exatamente como no enunciado
- ☐ `yarn typecheck` termina sem erros
- ☐ Não há `console.log` de teste, imports sem uso ou código comentado
- ☐ O arquivo `.env` **não** foi commitado

9.2 Como entregar

1. Trabalhe no **seu** repositório, criado no início pelo botão **Use this template** do GitHub.
2. Suba o seu código para ele: `git push` (direto na branch main, não precisa criar outra).
3. Envie o link do **seu** repositório pelo canal e no prazo informados pelos instrutores.
4. Se fez algum bônus ou tomou alguma decisão diferente, explique no README ou na mensagem de entrega.

9.3 Como vamos avaliar

Critério	O que olhamos
Funcionamento	O CRUD e as funcionalidades 7, 8 e 9 funcionam
Status HTTP	Cada resposta usa o status certo (201, 400, 404, 500...)
Validações	As regras de negócio são respeitadas e as mensagens de erro são claras
Organização	Separação entre controller e repository; rotas no padrão `/plantas`
Prisma	Model e migration corretos; uso adequado de `where` e do enum
Código limpo	Nomes claros, sem código morto, commits organizados

Os bônus contam pontos extras, mas não compensam requisitos obrigatórios faltando.

9.4 Travou? Como pedir ajuda

Travar faz parte e pedir ajuda não tira ponto. Antes, tente sozinho por uns 20 minutos: leia a mensagem de erro com calma, procure nos logs (`docker compose logs -f server`), volte ao exemplo de usuário e releia a parte 8 deste guia.

Ao pedir ajuda, mande:

- **O que você queria** que acontecesse;
- **O que aconteceu** (a mensagem de erro completa ou um print);
- **O que você já tentou**;
- O trecho de código envolvido.

Documentações que vão te ajudar: prisma.io/docs (Prisma Client e schema), expressjs.com/pt-br (Express) e developer.mozilla.org (status HTTP).

> 🌿 **Boa sorte e boas regas!**
> O objetivo deste desafio é aprender. Faça com calma, teste cada passo, pergunte quando travar e divirta-se construindo a API da Folhagem & Cia.

10 Glossário

Termo	Significado
API	O programa que recebe pedidos (requisições) e devolve respostas. O "garçom" entre o cliente e o banco.
Endpoint / rota	A combinação de método e URL que a API atende, como `GET /plantas`.
CRUD	Create, Read, Update, Delete: as quatro operações básicas com dados.
Requisição	O pedido que o cliente faz: método, URL, params, query string e body.
Resposta	O que a API devolve: um status HTTP e, geralmente, um corpo em JSON.
Status HTTP	Número que resume o resultado: 2xx sucesso, 4xx erro do cliente, 5xx erro do servidor.
JSON	Formato de texto para trocar dados: `{ "chave": valor }`.
Params	Parte variável da URL, como o `3` em `/plantas/3`. Lido com `req.params`.
Query string	O que vem depois do `?` na URL, usado para filtros. Lido com `req.query`.
Body	O corpo da requisição, com os dados enviados. Lido com `req.body`.
Express	Biblioteca do Node.js para criar servidores e rotas HTTP.
ORM / Prisma	Ferramenta que deixa você falar com o banco usando código TypeScript em vez de SQL.
Model	A definição de uma tabela no `schema.prisma`.
Enum	Tipo que só aceita uma lista fechada de valores, como `SOMBRA | MEIA_SOMBRA | SOL_PLENO`.
Migration	Arquivo que registra uma mudança na estrutura do banco. O "histórico de commits" do banco.
Controller	Camada que recebe a requisição, valida os dados e monta a resposta.
Repository	Camada que faz as consultas no banco.
Docker / container	Ferramenta que roda programas (a API, o banco) em ambientes isolados e iguais em qualquer máquina.
Hash	Transformação de mão única, usada para guardar senhas sem poder lê-las de volta (o bcrypt, por exemplo).
Seed	Script que preenche o banco com dados de exemplo.